Message Types in ShortP2P

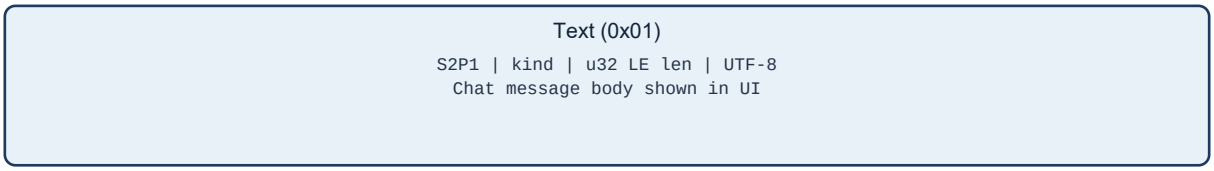
DRAFT - ChatWire and messenger-internal plaintext types. Examples are unencrypted application payloads (before P2P/server envelopes).

Author: Igor Evdokimov
Repository: https://github.com/IskraMessenger/ShortP2P
Created: 2026-11-11
Modified: 2026-11-11
Vector sources: docs/messages/diagrams/*.svg

Contents

Title page
Contents
1. Text — ChatWire kind 0x01
2. Image — ChatWire kind 0x02
3. File — ChatWire kind 0x03
4. Transfer offer — ChatWire kind 0x04
5. Transfer control — ChatWire kind 0x05
6. User info — ChatWire kind 0x06
7. Vocabulary offer — ChatWire kind 0x07
8. Vocabulary accept — ChatWire kind 0x08
9. Compressed text — ChatWire kind 0x09
10. Session probe — ChatWire kind 0x0A
11. Legacy UTF-8 text (no S2P1 magic)
12. Delivery ACK — messenger 0xAC
13. Delivery NACK — messenger 0xAD

1. Text — ChatWire kind 0x01



Unencrypted application plaintext (before session/server envelope).

Name

Text (ChatWireText)

Description

```
Human-readable chat text shown in the conversation UI.
Wire: magic ASCII "S2P1" (4) + kind 0x01 + text length u32 little-endian + UTF-8 bytes.
Source type: ChatWireText / ChatWireCodec.EncodeText.
```

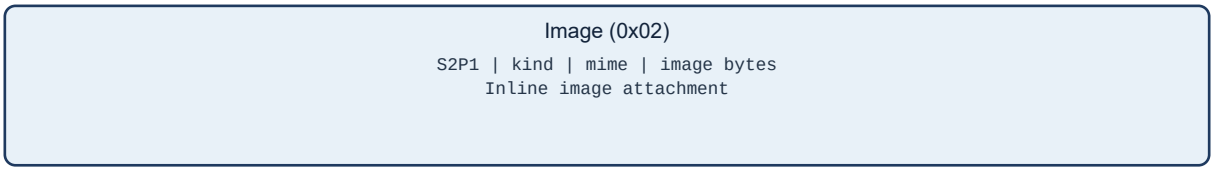
Unencrypted example

```
Logical: text = "Hello"

Hex (unencrypted):
  53 32 50 31  01  05 00 00 00  48 65 6C 6C 6F
  | S2P1   |kind| len=5 LE   |  H  e  l  l  o

Decoded: S2P1 | Text | "Hello"
```

2. Image — ChatWire kind 0x02



Unencrypted application plaintext (before session/server envelope).

Name

Image (ChatWireImage)

Description

```
Inline image attachment (bytes travel inside the encrypted chat channel).
Wire: S2P1 + 0x02 + mimeLen u16 LE + MIME UTF-8 + imageLen u32 LE + image bytes.
Source type: ChatWireImage / EncodeImage.
```

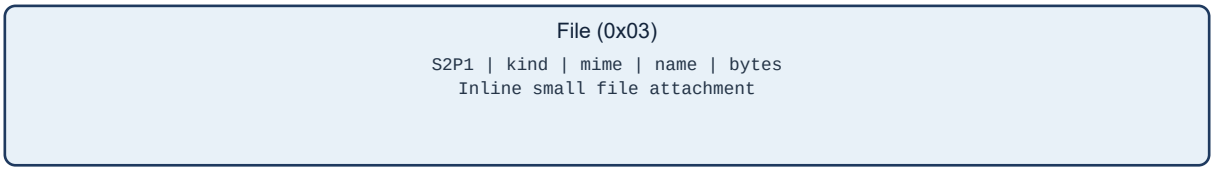
Unencrypted example

```
Logical: mimeType = "image/png", imageBytes = <4 demo bytes>

Hex (unencrypted, truncated image body):
  53 32 50 31  02
  09 00                          # mimeLen = 9
  69 6D 61 67 65 2F 70 6E 67     # "image/png"
  04 00 00 00                    # imageLen = 4
  89 50 4E 47                    # demo payload (not a full PNG)

Decoded: S2P1 | Image | mime=image/png | 4 bytes
```

3. File — ChatWire kind 0x03



Unencrypted application plaintext (before session/server envelope).

Name

File (ChatWireFile)

Description

```
Inline file attachment with original file name (small files in-band).
Wire: S2P1 + 0x03 + mimeLen u16 LE + MIME + nameLen u16 LE + file name UTF-8
      + fileLen u32 LE + file bytes. File name is Path.GetFileName-normalized.
Source type: ChatWireFile / EncodeFile.
```

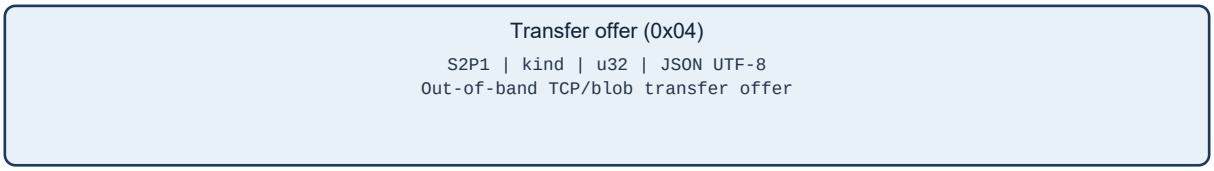
Unencrypted example

```
Logical: fileName = "note.txt", mimeType = "text/plain", fileBytes = "hi"

Hex (unencrypted):
  53 32 50 31  03
  0A 00  74 65 78 74 2F 70 6C 61 69 6E   # mime "text/plain"
  08 00  6E 6F 74 65 2E 74 78 74         # name "note.txt"
  02 00 00 00  68 69                     # len=2, "hi"

Decoded: S2P1 | File | text/plain | note.txt | "hi"
```

4. Transfer offer — ChatWire kind 0x04



Unencrypted application plaintext (before session/server envelope).

Name

Transfer offer (ChatWireTransferOffer)

Description

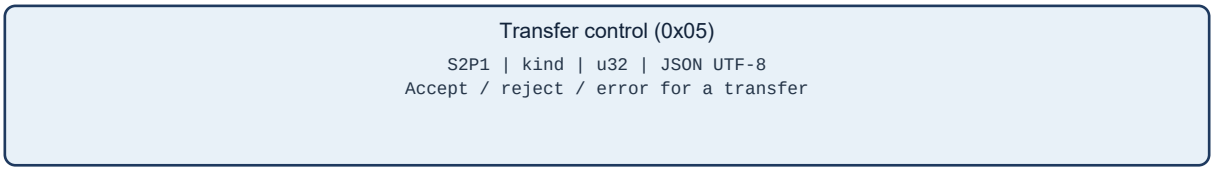
```
Offers a large out-of-band transfer (TCP host/port and/or server blob), negotiated
inside the encrypted chat. Payload after kind is JSON UTF-8 with length u32 LE.
Fields include TransferId, TransferToken, PayloadKind, FileName, MimeType,
SizeBytes, Host, Port, ExpiresUtcTicks, optional BlobId / TotalChunks / CiphertextSha256.
Source: ChatWireTransferOffer / EncodeTransferOffer.
```

Unencrypted example

```
Logical JSON (unencrypted body after S2P1|0x04|len):
  {
    "TransferId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "TransferToken": "tok-demo-001",
    "PayloadKind": "file",
    "FileName": "report.pdf",
    "MimeType": "application/pdf",
    "SizeBytes": 1048576,
    "Host": "192.168.1.20",
    "Port": 18700,
    "ExpiresUtcTicks": 638000000000000000,
    "BlobId": null,
    "TotalChunks": 0,
    "CiphertextSha256": null
  }

Frame: S2P1 | 0x04 | u32 LE(json length) | JSON UTF-8
```

5. Transfer control — ChatWire kind 0x05



Unencrypted application plaintext (before session/server envelope).

Name

Transfer control (ChatWireTransferControl)

Description

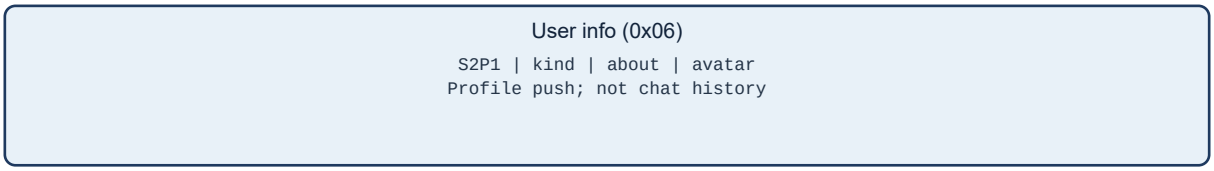
```
Control plane for an offered transfer (accept, reject, error, endpoint update).
Wire: S2P1 + 0x05 + u32 LE + JSON UTF-8.
Fields: Command, TransferId, TransferToken, Host, Port, ExpiresUtcTicks, ErrorCode.
Source: ChatWireTransferControl / EncodeTransferControl.
```

Unencrypted example

```
Logical JSON (unencrypted):
  {
    "Command": "accept",
    "TransferId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "TransferToken": "tok-demo-001",
    "Host": "192.168.1.20",
    "Port": 18700,
    "ExpiresUtcTicks": 638000000000000000,
    "ErrorCode": ""
  }

Frame: S2P1 | 0x05 | u32 LE(json length) | JSON UTF-8
```

6. User info — ChatWire kind 0x06



Unencrypted application plaintext (before session/server envelope).

Name

User info (ChatWireUserInfo)

Description

```
Pushes the sender's AboutMe text and optional avatar image to the peer.
Not stored as a normal chat bubble in history. Oversized avatars may be JPEG-compressed
to PeerProfileLimits.MaxAvatarBytes before encode.
Wire: S2P1 + 0x06 + aboutLen u16 LE + AboutMe UTF-8 + avatarLen u32 LE + avatar bytes.
Source: ChatWireUserInfo / EncodeUserInfo.
```

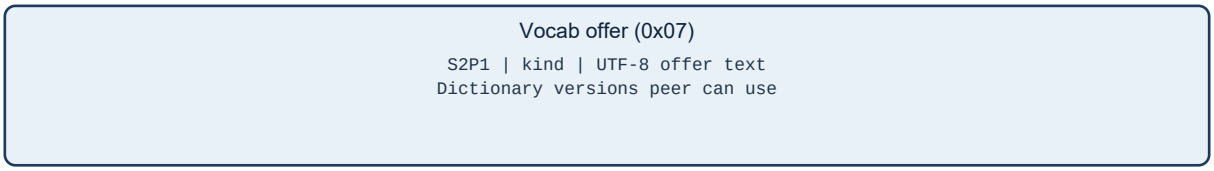
Unencrypted example

```
Logical: aboutMe = "Hi", avatar = empty

Hex (unencrypted):
  53 32 50 31  06
  02 00              # aboutLen = 2
  48 69              # "Hi"
  00 00 00 00        # avatarLen = 0

Decoded: S2P1 | UserInfo | about="Hi" | no avatar
```

7. Vocabulary offer — ChatWire kind 0x07



Unencrypted application plaintext (before session/server envelope).

Name

Vocabulary offer (ChatWireVocabOffer)

Description

```
Announces which compression dictionaries (language + version + BLAKE3 hex) this peer
can use. Format of the UTF-8 body (VocabularyNegotiation):
  LANG:ver,hash;ver,hash|LANG2:ver,hash
Example languages are upper-case codes such as RU, EN.
Source: ChatWireVocabOffer / EncodeVocabOffer.
```

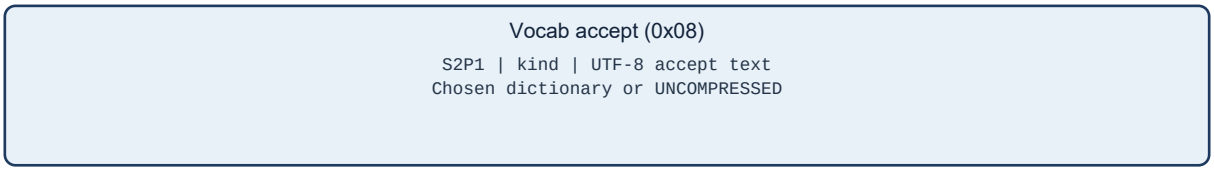
Unencrypted example

```
Logical OfferText:
  RU:1,aabbccddeeff0011|EN:1,1122334455667788

Hex (unencrypted, first bytes):
  53 32 50 31  07  <u32 LE length>  <UTF-8 OfferText...>

Decoded: S2P1 | VocabOffer | RU dict v1 + EN dict v1 hashes
```

8. Vocabulary accept — ChatWire kind 0x08



Unencrypted application plaintext (before session/server envelope).

Name

Vocabulary accept (ChatWireVocabAccept)

Description

```
Selects dictionaries for compressed text, or UNCOMPRESSED per language.
Accept text format: LANG:ver,hash|LANG2:UNCOMPRESSED
Source: ChatWireVocabAccept / EncodeVocabAccept.
```

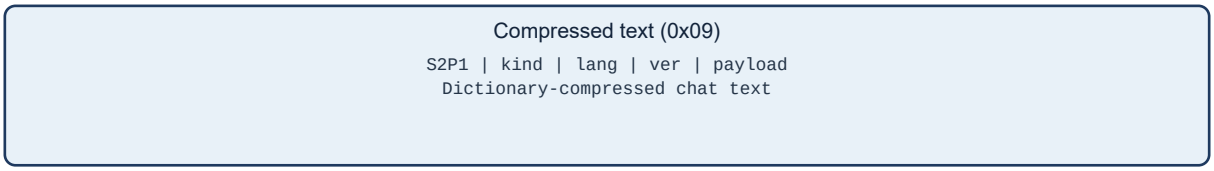
Unencrypted example

```
Logical AcceptText:
  RU:1,aabbccddeeff0011|EN:UNCOMPRESSED

Frame: S2P1 | 0x08 | u32 LE | UTF-8 AcceptText

Decoded: use RU dictionary v1; leave EN uncompressed
```

9. Compressed text — ChatWire kind 0x09



Unencrypted application plaintext (before session/server envelope).

Name

Compressed text (ChatWireCompressedText)

Description

```
Chat text encoded with an agreed vocabulary pack instead of raw UTF-8.
Wire: S2P1 + 0x09 + langLen u8 + language UTF-8 (upper) + dictVersion u16 LE
      + payloadLen u32 LE + compressed payload bytes.
Source: ChatWireCompressedText / EncodeCompressedText.
```

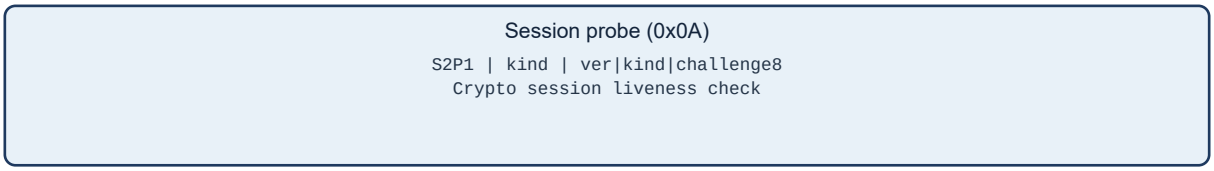
Unencrypted example

```
Logical: language = "RU", dictVersion = 1, payload = AA BB (demo)

Hex (unencrypted):
  53 32 50 31  09
  02  52 55           # langLen=2, "RU"
  01 00               # dictVersion = 1 LE
  02 00 00 00         # payloadLen = 2
  AA BB

Decoded: S2P1 | CompressedText | RU v1 | 2-byte payload
```

10. Session probe — ChatWire kind 0x0A



Unencrypted application plaintext (before session/server envelope).

Name

Session probe (ChatWireSessionProbe)

Description

```
Liveness / crypto-session check after handshake; not shown as chat history.
Body (after kind + u32 length): version=1 | probeKind | challenge[8].
probeKind: 1 = send (KindSend), 2 = response (KindResponse).
Sized so one messenger chunk stays within a small encrypted radio budget (~80 B).
Source: ChatWireSessionProbe / SessionCryptoProbe / EncodeSessionProbe.
```

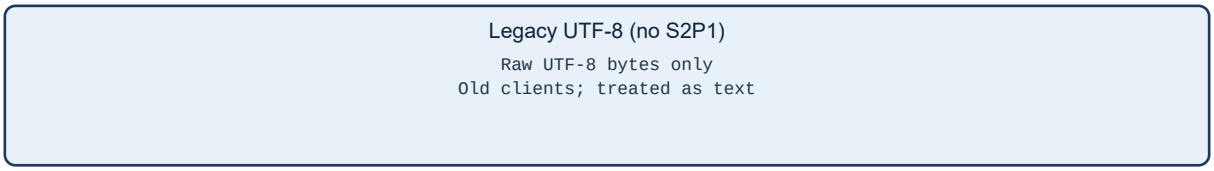
Unencrypted example

```
Logical: probeKind = 1 (send), challenge = 01..08

Hex (unencrypted):
  53 32 50 31  0A  0A 00 00 00
  01              # body version
  01              # KindSend
  01 02 03 04 05 06 07 08

Decoded: S2P1 | SessionProbe | send | challenge 0102030405060708
```

11. Legacy UTF-8 text (no S2P1 magic)



Unencrypted application plaintext (before session/server envelope).

Name

Legacy plaintext UTF-8

Description

```
Older peers may send chat text without the S2P1 frame. If ChatWireCodec.TryParse fails
and the payload does not look like a framed wire, the messenger treats the bytes as UTF-8 text.
Still carried inside the encrypted P2P/server envelope; only the application framing is absent.
```

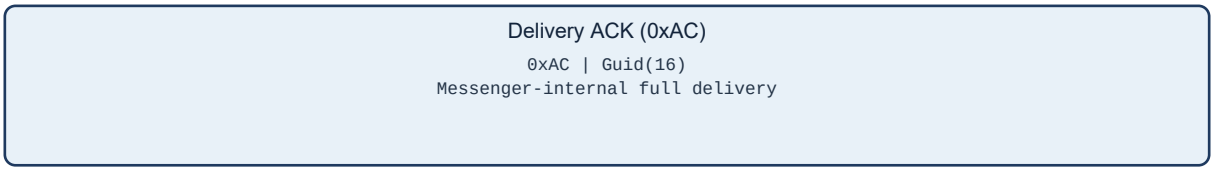
Unencrypted example

```
Logical: "Hello"

Hex (unencrypted application payload):
  48 65 6C 6C 6F

Decoded: UTF-8 "Hello" (no S2P1 / kind byte)
```

12. Delivery ACK — messenger 0xAC



Unencrypted application plaintext (before session/server envelope).

Name

Delivery ACK (DeliveryAckCodec)

Description

```
Messenger-internal receipt: the peer fully reassembled all chunks of a message.
Not a ChatWire kind; plaintext starts with marker 0xAC. Encrypted under the same
P2P session as chat (Scheme 2), then prefixed 0x02 on the wire.
Layout: 0xAC + messageId Guid (16 bytes, Guid.TryWriteBytes order). Total 17 bytes.
```

Unencrypted example

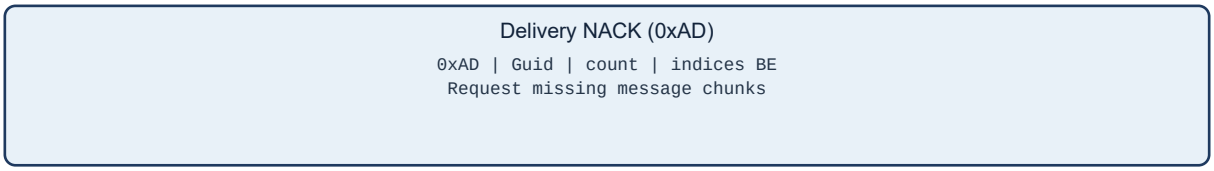
```
Logical: messageId Guid bytes as written by .NET Guid.TryWriteBytes

Hex (unencrypted messenger plaintext):
  AC  <16 Guid bytes>

Example pattern:
  AC 00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF
  (illustrative Guid layout; use actual TryWriteBytes in implementations)

Decoded: ACK for messageId
```

13. Delivery NACK — messenger 0xAD



Unencrypted application plaintext (before session/server envelope).

Name

Delivery NACK (DeliveryNackCodec)

Description

```
Messenger-internal request to retransmit missing chunks of a multi-chunk message.
Layout: 0xAD + messageId Guid(16) + count u16 big-endian + count × int32 big-endian indices.
Encrypted like ACK under the P2P session.
```

Unencrypted example

```
Logical: messageId = <Guid>, missing chunk indices = [0, 2]

Hex (unencrypted):
  AD  <16 Guid bytes>
  00 02                 # count = 2 (BE)
  00 00 00 00           # index 0 (BE)
  00 00 00 02           # index 2 (BE)

Decoded: NACK — please resend chunks 0 and 2 for messageId
```